# UGANDACHRISTIANUNIVERSITY

Faculty of Engeering Design and Technology

DATABASE APPLICATION AND DESIGN

LIBRARY RESEARCH, READING AND GROUP PRESENTATION ACTIVITY

TOPIC:

Database Normalisation and Database Constraints

## GROUP MEMBERS

| No. | Name | Reg. No. |
|---|---|---|
| 1 | AMANYA AARON | M26B13/006 |
| 2 | NABUKENYA SARAH | M26B13/031 |
| 3 | MIREMBE NORAH | M26B13/025 |
| 4 | AMPUMUZA ESTHER | M26B13/086 |
| 5 | NAKAFEERO HADIJAH KARAMAGI | M26B13/032 |
| 6 | WADEJJE AARON DAVID | M26B13/042 |
| 7 | NOKUKUNDAKWE MARTHA | M26B13/035 |

## 1 Introduction

This report presents our group's findings from a library research activity on database normalisation and database constraints. We visited the UCU Library, consulted several textbooks on database design, and used what we learnt to prepare this report. The report explains normalisation, the three anomalies it solves, and how tables are converted to 1NF, 2NF, and 3NF. It also covers database constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT, and how they keep data accurate. At the end, we compare the two concepts and explain why a database designer needs both.

We found this topic challenging at first because the textbook explanations were quite formal. But after discussing the examples among ourselves, the ideas became much clearer.

## 2 Library Research Requirement

In accordance with the activity requirements, our group visited the UCU Library and located the following textbooks. These were the editions actually available on the shelves at the time of our research.

| No. | Author(s) | Book Title | Edition | Year | Relevant Chapter |
|---|---|---|---|---|---|
| 1 | Connolly, T. & Begg, C. | Database Systems: A Practical Approach to Design, Implementation, and Management | 6th | 2015 | Ch. 4 (Normalisation), Ch. 5 (Constraints) |
| 2 | Elmasri, R. & Navathe, S. B. | Fundamentals of Database Systems | 7th | 2016 | Ch. 14 (Normalisation), Ch. 4 (Constraints) |
| 3 | Silberschatz, A., Korth, H. F. & Sudarshan, S. | Database System Concepts | 7th | 2019 | Ch. 7 (Normalisation), Ch. 3 (Constraints) |
| 4 | Ramakrishnan, R. & Gehrke, J. | Database Management Systems | 3rd | 2003 | Ch. 19 (Normalisation), Ch. 3 (Constraints) |

Table 1: Textbooks consulted during library research

We found Connolly & Begg the most useful for understanding anomalies, while Elmasri & Navathe explained the higher normal forms more clearly. Silberschatz was helpful for the SQL examples.

# 3 Part A: Research on Database Normalisation

## 3.1 Meaning of Normalisation

Normalisation is the process of organising the data in a relational database so that redundancy is reduced and data dependencies are properly controlled. In simple terms, it is the act of breaking a large, messy table into smaller, well-structured tables and linking them using relationships. Instead of storing the same piece of information in many places, we store it once and refer to it from other tables.

Why normalisation is important in relational database design:

- It removes data redundancy, the same data is not stored in multiple places.
- It ensures data consistency, meaning that if data is stored once, it cannot contradict itself.
- It prevents anomalies like insertion, update, and deletion problems.
- It makes maintenance easier because changes are made in one place only.
- It saves storage space, so fewer duplicates mean less wasted space.

- It improves data integrity so that the database reflects the real world accurately.

The problems normalisation attempts to solve:

Normalisation exists mainly to solve the problems caused by storing too much unrelated data in one table. These problems include data redundancy, insertion anomalies, update anomalies, and deletion anomalies. When a table contains data about more than one entity (for example, students and courses in one table), problems arise whenever we try to insert, update, or delete records.

Example in our own words:

Imagine a school that keeps all its records in one big table: student name, student phone, course name, lecturer name, and lecturer office. If a lecturer changes office, every row for every student taught by that lecturer must be updated. If one row is missed, the database now contains conflicting information. Normalisation solves this by separating students, courses, and lecturers into their own tables, so each fact is stored only once.

## 3.2 Database Anomalies

An anomaly is a problem or error that occurs in a database because of poor design. There are three main types.

### 3.2.1 Insertion Anomaly

- This means that an insertion anomaly occurs when we cannot add new data to the database without also adding unrelated data.
- Here the practical example is, Suppose we have one table containing student and course information. If we want to add a new course that has no students yet, we cannot do so because the table requires a student ID for every row.
- How poor design causes it: The table mixes two different entities (students and courses), so one cannot exist without the other in the table.
- Therefore normalisation helps by separating courses into their own table, we can insert a new course without needing a student.

### 3.2.2 Update Anomaly

- This means that an update anomaly occurs when we need to update the same piece of data in multiple rows, and failing to update all of them creates inconsistency.
- The practical example is, If a lecturer's office number is stored in every row of a registration table, changing the office means updating many rows. If one row is missed, the database shows two different offices for the same lecturer.

- How poor design causes it: Data redundancyand this means the same fact is stored many times.

- normalisation helps by storing the lecturer's office in a single Lecturer table means we update it only once.

### 3.2.3 Deletion Anomaly

- A deletion anomaly occurs when deleting a record unintentionally removes other useful information.

- Here the practical example is,If a student is the only person enrolled in a particular course and we delete that student's record, the course information disappears too.

- How poor design causes it: The table combines unrelated data, so deleting one entity accidentally deletes another.

- normalisation helps when students, courses, and enrolments are stored in separate tables, deleting a student does not affect the course.

## 3.3 Functional Dependency

A functional dependency is a relationship between two attributes where one attribute uniquely determines the other. We write it as A → B, which reads "A functionally determines B." This means that if we know the value of A, we can determine the value of B.

- Determinant: The attribute on the left side of the arrow (A). It is the attribute doing the determining.

- Dependent: The attribute on the right side of the arrow (B). It is the attribute being determined.

How functional dependencies are identified during database design:

Functional dependencies are identified by studying the business rules and the meaning of the data. For example, if the university says "each student has one student ID," then StudentID → StudentName. We ask questions like: "Given this attribute, can we determine only one value of that attribute?" If yes, a functional dependency exists.

Examples:

1. StudentID → StudentName, DateOfBirth, Phone
   Knowing a student's ID tells us exactly one name, one date of birth, and one phone number.

2. CourseCode → CourseTitle, CreditHours
   Knowing a course code tells us exactly one course title and one number of credit hours.

## 3.4 First Normal Form (1NF)

Requirements for a relation to be in 1NF:

1. Every column must contain atomic (indivisible) values.
2. Each column must have a unique name.
3. Each row must be unique (usually ensured by a primary key).
4. There must be no repeating groups or multi-valued columns.

Example of a table NOT in 1NF:

| StudentID | StudentName | Courses |
|---|---|---|
| 101 | Alice | DB, Java |
| 102 | Bob | Networks, DB, Java |

Table 2: Table violating 1NF (multi-valued “Courses” column)

The “Courses” column contains multiple values in one cell. This violates 1NF.

Transformation into 1NF:

| StudentID | StudentName | Course |
|---|---|---|
| 101 | Alice | DB |
| 101 | Alice | Java |
| 102 | Bob | Networks |
| 102 | Bob | DB |
| 102 | Bob | Java |

Table 3: Table in 1NF after removing multi-valued column

Now every column contains only atomic values, and each row is unique.

## 3.5 Second Normal Form (2NF)

Requirements for 2NF:

1. The relation must already be in 1NF.
2. There must be no partial dependency, no non-key attribute should depend on only part of a composite primary key.

Partial Dependency:

A partial dependency exists when a non-key attribute depends on only a portion of a composite primary key instead of the whole key. For example, if the primary key is (StudentID, CourseID) and StudentName depends only on StudentID, that is a partial dependency.

Why partial dependencies cause database problems:

Partial dependencies cause data redundancy because the same value (e.g., StudentName) is repeated for every course the student takes. They also cause update anomalies, if a student changes their name, all rows must be updated. Insertion and deletion anomalies can also occur.

Example: Converting 1NF to 2NF:

Consider this 1NF table:

| StudentID | CourseID | StudentName | CourseTitle |
|---|---|---|---|
| 101 | C1 | Alice | Database |
| 101 | C2 | Alice | Java |
| 102 | C1 | Bob | Database |

Table 4: 1NF table with partial dependencies

Composite primary key: (StudentID, CourseID)

Problems:

- StudentName depends only on StudentID (partial dependency).
- CourseTitle depends only on CourseID (partial dependency).

Convert to 2NF by splitting:

| StudentID | StudentName |
|---|---|
| 101 | Alice |
| 102 | Bob |

Table 5: Student table

| Co |
|---|
| |

| StudentID | CourseID |
|---|---|
| 101 | C1 |
| 101 | C2 |
| 102 | C1 |

Table 6: Course table

Table 7: Enrolment table

All partial dependencies are now removed.

## 3.6 Third Normal Form (3NF)

Requirements for 3NF:

1. The relation must already be in 2NF.
2. There must be no transitive dependency, no non-key attribute should depend on another non-key attribute.

Transitive Dependency:

A transitive dependency exists when a non-key attribute depends on another non-key attribute, which in turn depends on the primary key. If A → B and B → C, then A → C is a transitive dependency.

Example: Converting to 3NF:

| StudentID | CourseID | LecturerName | LecturerOffice |
|---|---|---|---|
| 101 | C1 | Dr. Smith | A101 |
| 102 | C1 | Dr. Smith | A101 |

Table 8: 2NF table with transitive dependency

Consider this 2NF table:

Here, LecturerOffice depends on LecturerName, which depends on CourseID. This is a transitive dependency.

Convert to 3NF by splitting:

| LecturerName | LecturerOf | ce( |
|---|---|---|
| Dr. Smith | A101 | |

Table 9: Lecturer table

| StudentID | CourseID |
|---|---|
| 101 | C1 |
| 102 | C1 |

Table 10: Course table

Table 11: Enrolment table

The transitive dependency is now removed.

## 3.7 Higher Normal Forms

- BCNF (Boyce-Codd Normal Form): A stricter version of 3NF. Every determinant must be a candidate key. It handles certain anomalies that 3NF cannot.
- 4NF (Fourth Normal Form): Deals with multi-valued dependencies, ensuring that a table does not contain two or more independent multi-valued facts about the same entity.
- 5NF (Fifth Normal Form): Deals with join dependencies, ensuring that data can be reconstructed from smaller tables without loss of information.

Why higher normal forms exist:

Higher normal forms exist because 1NF, 2NF, and 3NF cannot handle all possible anomalies, especially in complex databases with multi-valued attributes or complex relationships. They provide stricter rules to eliminate rare but real problems.

## 3.8 Practical Application

Scenario: University Student Registration System Step 1:
Poorly Designed Table (Unnormalised):

| StudentID | StudentName | Courses | Lecturer | LecturerOffice |
|---|---|---|---|---|
| 101 | Alice | DB, Java | Dr. Smith, Dr. Jones | A101, B202 |
| 102 | Bob | DB | Dr. Smith | A101 |

Table 12: Unnormalised table

This table violates 1NF because “Courses” and “Lecturer” contain multiple values.

Step 2: Convert to 1NF:

| StudentID | StudentName | Course | Lecturer | LecturerOffice |
|---|---|---|---|---|
| 101 | Alice | DB | Dr. Smith | A101 |
| 101 | Alice | Java | Dr. Jones | B202 |
| 102 | Bob | DB | Dr. Smith | A101 |

Table 13: Table in 1NF

Step 3: Convert to 2NF:

Remove partial dependencies. StudentName depends only on StudentID; Lecturer depends only on Course.

| StudentID | StudentName |
|---|---|
| 101 | Alice |
| 102 | Bob |

Table 14: Student table

| StudentID | Course |
|---|---|
| 101 | DB |
| 101 | Java |
| 102 | DB |

Table 15: Course table

Table 16: Enrolment table

Step 4: Convert to 3NF:

Remove transitive dependency. LecturerOffice depends on Lecturer.

| Lecturer | LecturerOffice |
|---|---|
| Dr. Smith | A101 |
| Dr. Jones | B202 |

Table 17: Lecturer table

| Course | Lecturer |
|---|---|
| DB | Dr. Smith |
| Java | Dr. Jones |

Table 18: Course table

The database is now in 3NF, with no redundancy and no anomalies.

# 4 Part B: Research on Database Constraints

## 4.1 What is a Database Constraint?

A constraint is a rule or restriction placed on the data in a database to ensure that only valid and accurate data is stored. Constraints are enforced automatically by the Database Management System (DBMS), so even if a user tries to enter wrong data, the database will reject it.

Why constraints are important:

- They enforce business rules at the database level.
- They prevent invalid, incomplete, or duplicate data.
- They maintain consistency across related tables.
- They protect the database from accidental or malicious errors.

How constraints help maintain data integrity:

Data integrity means the data in the database is accurate, consistent, and reliable. Constraints help by ensuring that:

- Every row is uniquely identifiable (PRIMARY KEY).
- Relationships between tables remain valid (FOREIGN KEY).
- No duplicate values appear where they shouldn't (UNIQUE).
- Important columns are never left empty (NOT NULL).
- Values fall within acceptable ranges (CHECK).
- Missing values are automatically filled with sensible defaults (DEFAULT).

## 4.2 PRIMARY KEY Constraint

What a primary key is:

A primary key is a column (or combination of columns) that uniquely identifies every row in a table.

Characteristics of a primary key:

- It must be unique for every row.
- It cannot be NULL.
- Each table can have only one primary key.
- It should be stable (rarely changes).

Why a primary key is important:

It ensures that every record can be uniquely identified and referenced, which is essential for relationships between tables.

SQL Example:

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50)
);
```

Listing 1: PRIMARY KEY constraint example

## 4.3 FOREIGN KEY Constraint

What a foreign key is:

A foreign key is a column (or set of columns) in one table that refers to the primary key of another table.

How it establishes relationships between tables:

It creates a link between two tables. For example, an Enrolment table can have a StudentID foreign key that points to the Students table, showing which student is enrolled.

Referential integrity:

Referential integrity means that the value in the foreign key column must either match a value in the referenced table or be NULL. This prevents orphan records (records that refer to nonexistent data).

SQL Example:

```
CREATE TABLE Enrolment (
    StudentID INT,
    CourseID INT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID) );
```

Listing 2: FOREIGN KEY constraint example

## 4.4 UNIQUE Constraint

What the UNIQUE constraint does:

It ensures that all values in a column (or combination of columns) are different from each other.

How it differs from a PRIMARY KEY:

| PRIMARY KEY | UNIQUE |
|---|---|
| Only one per table<br>Cannot be NULL<br>Uniquely identifies rows | Multiple allowed per table<br>Can allow NULL (depending on DBMS)<br>Simply prevents duplicates |

Table 19: Difference between PRIMARY KEY and UNIQUE

Example:

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);
```

Listing 3: UNIQUE constraint example

Here, two employees cannot share the same email address.

## 4.5 NOT NULL Constraint

Not Null ensures that a column cannot be left empty and a value must be provided for every row.

It is usefull because some data is essential. For example, a student must have a name. Allowing NULL in such a column would create incomplete records.

Practical Example:

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50) NOT NULL
);
```

Listing 4: NOT NULL constraint example

## 4.6 CHECK Constraint

It limits the values that can be entered into a column based on a logical condition.

The practical Example is preventing a negative employee salary

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Salary DECIMAL(10,2) CHECK (Salary >= 0)
);
```

Listing 5: CHECK constraint example

Any attempt to insert a negative salary will be rejected by the DBMS.

## 4.7 DEFAULT Constraint

It provides a value that is automatically used when no value is specified for a column.

It is useful when a column should have a standard value unless the user explicitly changes it. For example, a new order should default to today's date.

Example:

```
CREATE TABLE Orders (
OrderID INT PRIMARY KEY,
    OrderDate DATE DEFAULT CURRENT_DATE
);
```

Listing 6: DEFAULT constraint example

## 4.8 Combining Constraints

Example table containing at least four different constraints:

```
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) CHECK (Price > 0),
    Stock INT DEFAULT 0,
    Barcode VARCHAR(50) UNIQUE
);
```

Listing 7: Table with multiple constraints

Explanation of each constraint:

| Constraint | Purpose |
|---|---|
| PRIMARY KEY (ProductID) | Uniquely identifies each product. |
| NOT NULL (ProductName) | Ensures every product has a name. |
| CHECK (Price > 0) | Ensures the price is always positive. |
| DEFAULT (Stock = 0) | Sets stock to 0 if no value is given. |
| UNIQUE (Barcode) | Ensures no two products share a barcode. |

Table 20: Purpose of each constraint

# 5 Comparison Task: Normalisation vs. Constraints

1. What problem does normalisation solve?

   Normalisation solves the problem of poor database structure. It removes data redundancy and eliminates insertion, update, and deletion anomalies by organising data into well-structured tables.

2. What problem do constraints solve?

   Constraints solve the problem of invalid or inaccurate data. They enforce rules so that only correct, complete, and consistent data is stored in the database.

3. How are they different?

   Normalisation is a design process, it restructures tables and relationships. Constraints are rules, they restrict what data can be entered. Normalisation works at the level of table structure, while constraints work at the level of individual data values.

4. How do they complement each other?

   Normalisation creates a clean structure, and constraints protect that structure by ensuring only valid data enters it. A normalised database without constraints can still accept wrong data; a database with constraints but poor structure can still suffer from redundancy and anomalies. Together, they provide both structural efficiency and data integrity.

5. Can a well-normalised database still contain invalid data? Explain.

   Yes. Normalisation only addresses redundancy and anomalies caused by poor table structure. It does not check whether a salary is negative, whether an email is duplicated, or whether a name is missing. Without constraints, a well-normalised database can still contain invalid data. For example, a properly normalised Employees table could still accept a negative salary or a NULL name if no constraints are defined.

6. Why should a database designer consider both normalisation and constraints? A database designer must consider both because they solve different but equally important problems. Normalisation ensures the database is efficient and free from redundancy, while constraints ensure the data is accurate and reliable. Using only one would leave the database vulnerable either to structural problems or to invalid data. Both are essential for a highquality database system.

# 6 Conclusion

This research activity has helped us understand that database design is not just about creating tables, it is about creating the right tables and protecting them with the right rules. Normalisation ensures that data is stored efficiently, without redundancy, and without anomalies. Constraints ensure that the data stored is accurate, complete, and consistent.

We have learned that normalisation and constraints are not competing ideas; they work together. A well-designed database needs both. Normalisation builds the structure, and constraints guard it. Together, they produce a database that is reliable, maintainable, and trustworthy.